DESARROLLO ORIENTADO A OBJETOS [DOPO]
2026-2
Ciclo 5

Materia: Desarrollo orientado a objetos
Grupo: 7

Integrantes

Jhon Eduard Duran Ceballos
ID: 1000100325
Correo: jhon.duran-c@mail.escuelaing.edu.co

Andres Steven Oyuela Mendez
ID: 1000109048
Correo: andres.oyuela-m@mail.escuelaing.edu.co

1. Introducción

El presente documento detalla la fase final y de cierre del simulador interactivo SlotMachine. En esta última etapa, el objetivo principal es evaluar y perfeccionar la calidad del simulador mediante el uso de herramientas modernas de desarrollo orientadas a la industria. Para ello, el proyecto migra formalmente al IDE VS Code y se somete a rigurosos análisis estáticos y dinámicos para garantizar el cumplimiento de estándares de codificación, cobertura de pruebas y reglas de calidad en Java.

2. Definición y Planificación de Mini-Ciclos de Cierre

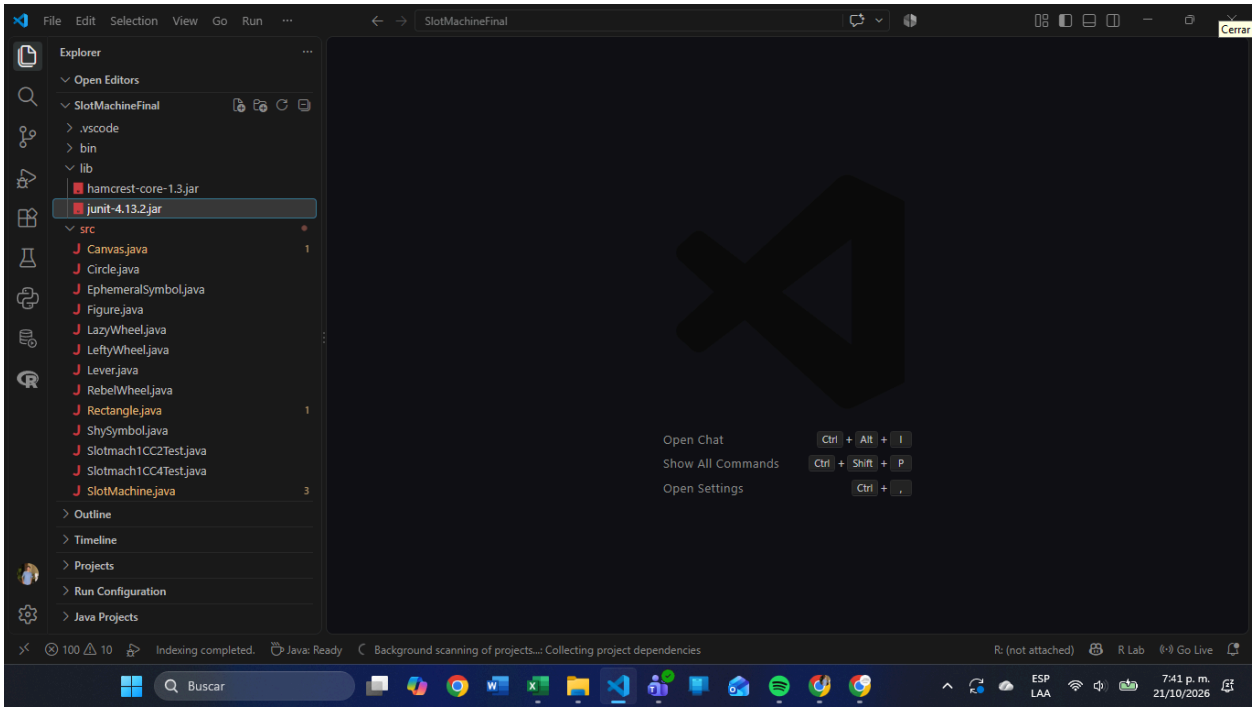
MINI-CICLO 1: Migración y Configuración del Entorno (VS Code)

Estado: Completado (100%).

Objetivo: Migrar la totalidad del proyecto SlotMachine al IDE VS Code, asegurando que todas las clases y dependencias compilen correctamente fuera de BlueJ.

Acciones realizadas:

- Configuración del entorno de trabajo en VS Code.
- Implementación y formateo de código siguiendo los estándares de codificación y documentación de Java.



MINI-CICLO 2: Análisis Dinámico y Cobertura (JaCoCo)

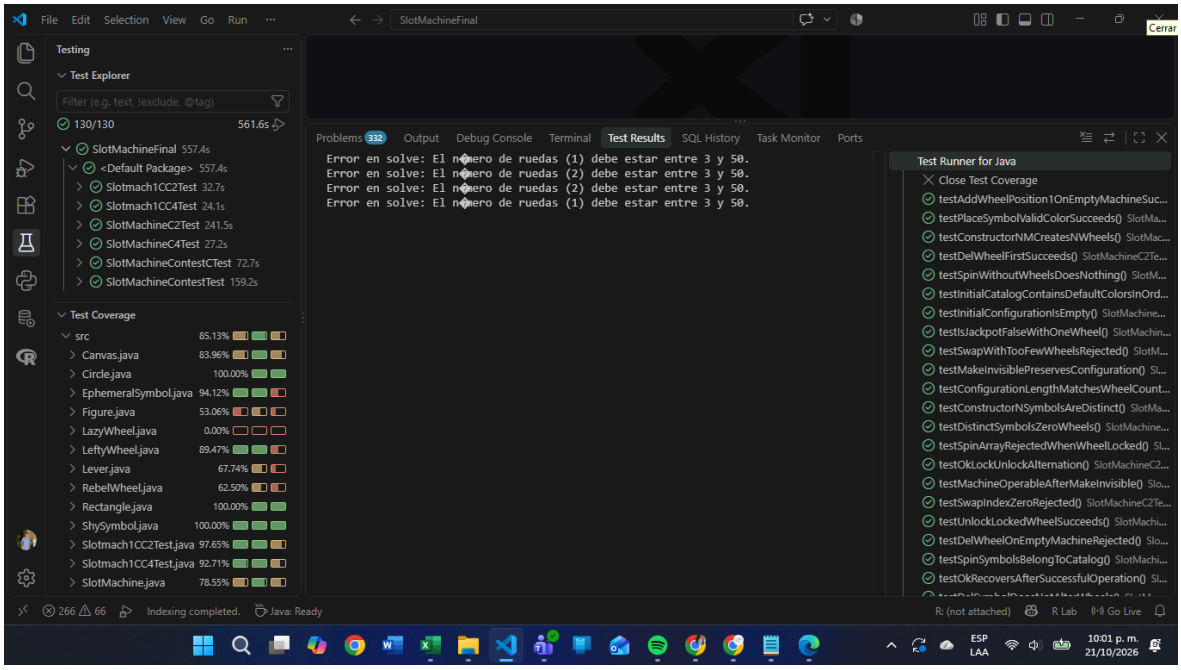
Estado: Completado (100%).

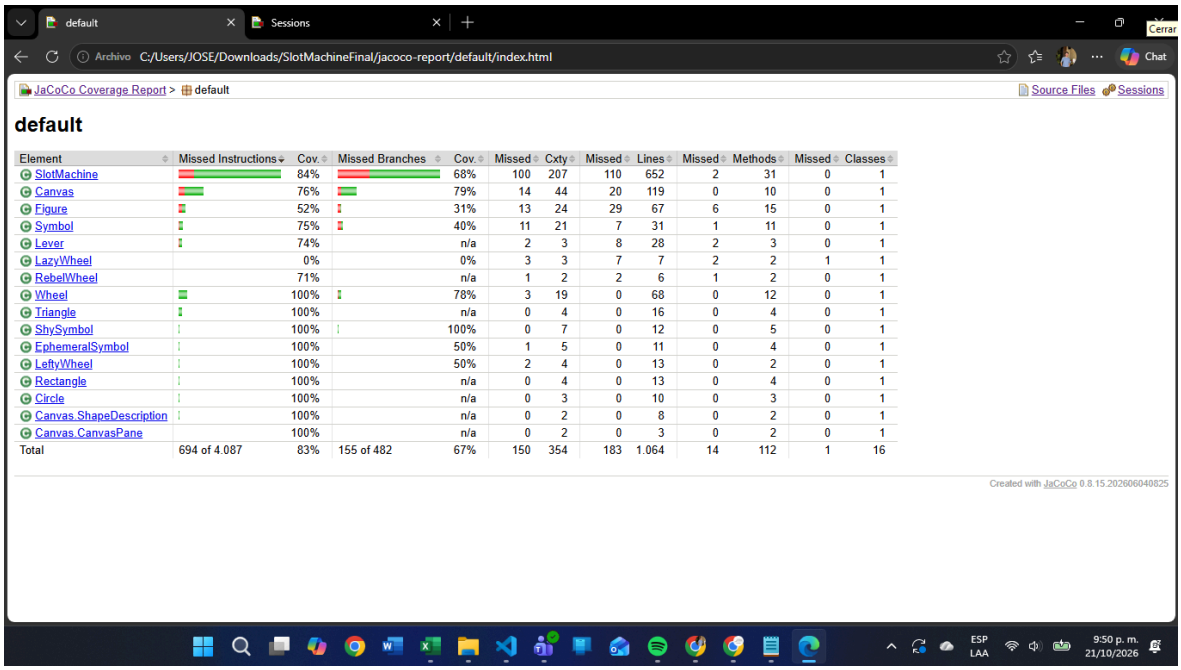
Objetivo: Generar el informe de análisis dinámico y alcanzar la meta de más de 75% de cubrimiento de código.

Acciones realizadas:

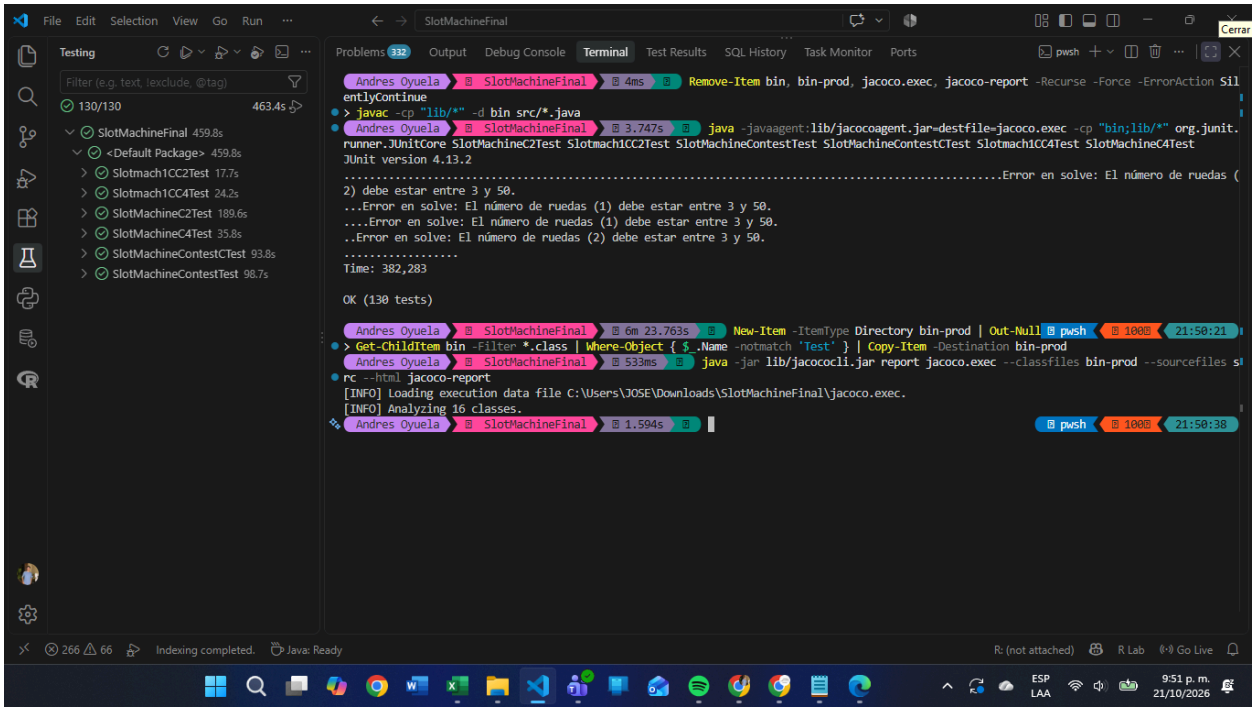
- Ejecución de JaCoCo para generar el resultado inicial.
- Refactorización y adición de casos de prueba para cubrir ramas de código faltantes.
- Generación del informe final superando la meta del 75% de cobertura.

ANTES del refactor y agregar las pruebas SlotMachineC5Test y Slotmach1CC5Test

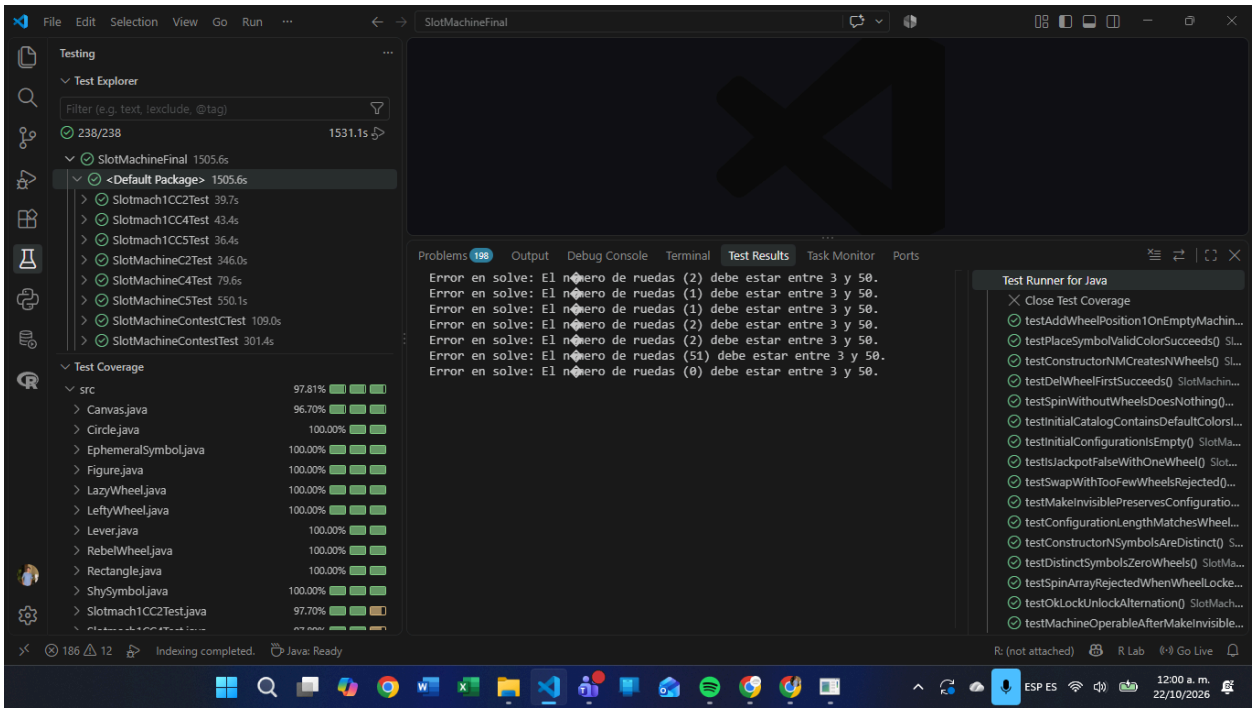


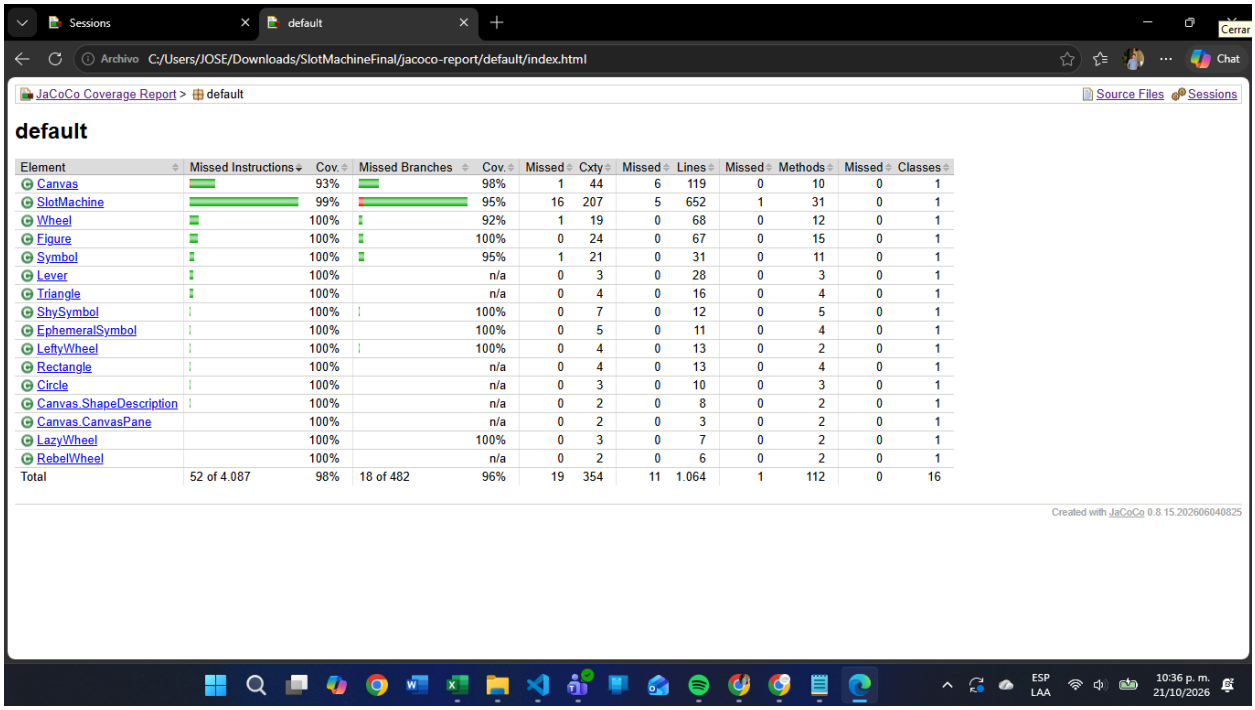


Proceso para crear el archivo JaCoCo de manera visual (Se encuentra en archivo txt “Pasos para el creación del reporte de JaCoCo”)

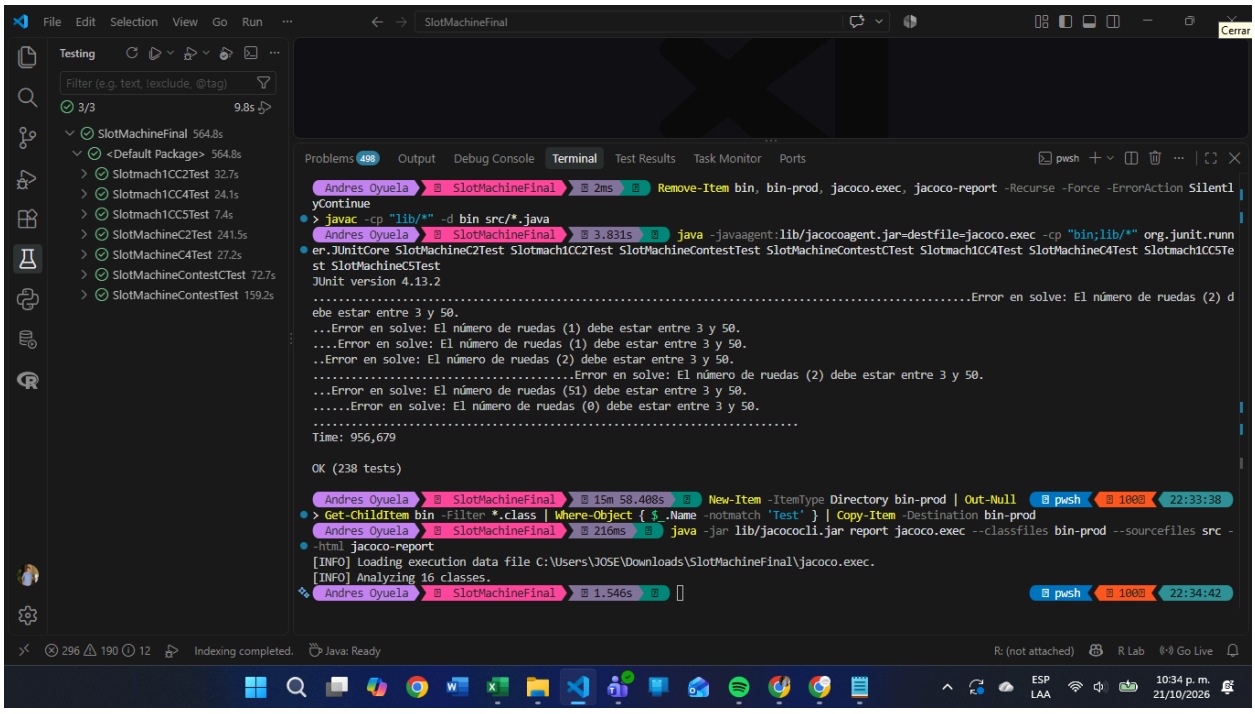


DESPUES del refactor y agregar las pruebas SlotMachineC5Test y Slotmach1CC5Test





Proceso para crear el archivo JaCoCo de manera visual (Se encuentra en archivo txt “Pasos para el creación del reporte de JaCoCo”)



MINI-CICLO 3: Análisis Estático y Calidad (PMD)

Estado: Completado (100%).

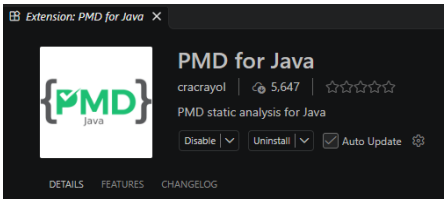
Objetivo: Ejecutar PMD para auditar la calidad del código y cumplir con todas las reglas de prioridad alta (rojas).

Acciones realizadas:

- Generación del informe estático inicial con PMD.
- Corrección de vulnerabilidades, variables no utilizadas, optimización de importaciones y refactorización de métodos complejos en la clase SlotMachine y sus dependencias.
- Generación del informe final evidenciando cero advertencias de prioridad alta.

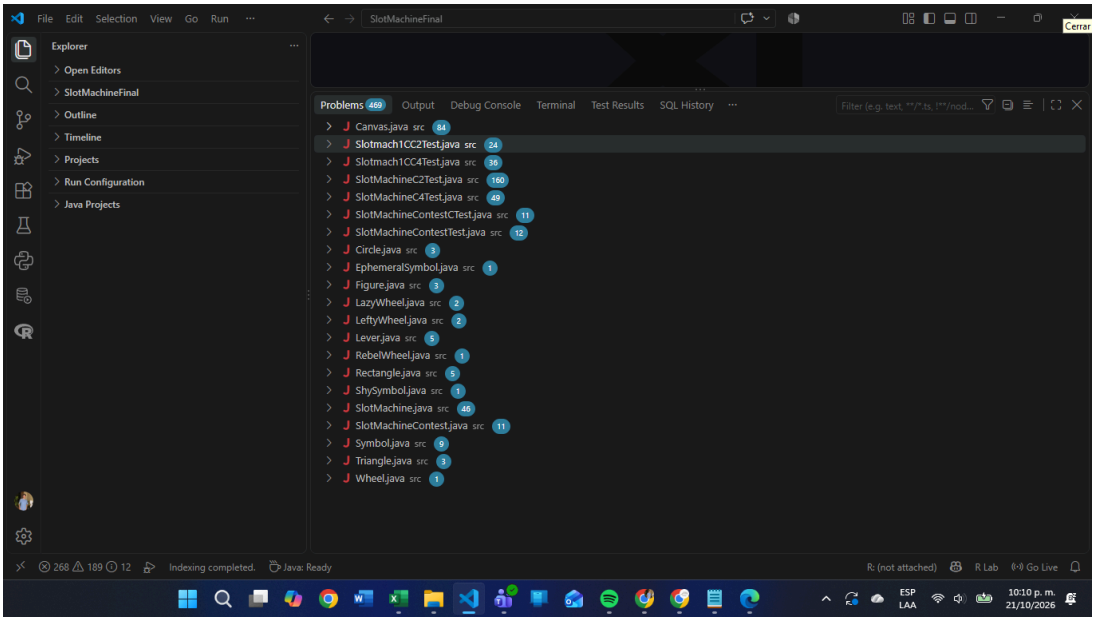
Proceso para instalar la auditoria para las buenas prácticas en el còdigo

1. El proceso para acceder es directamente desde visual instalar:

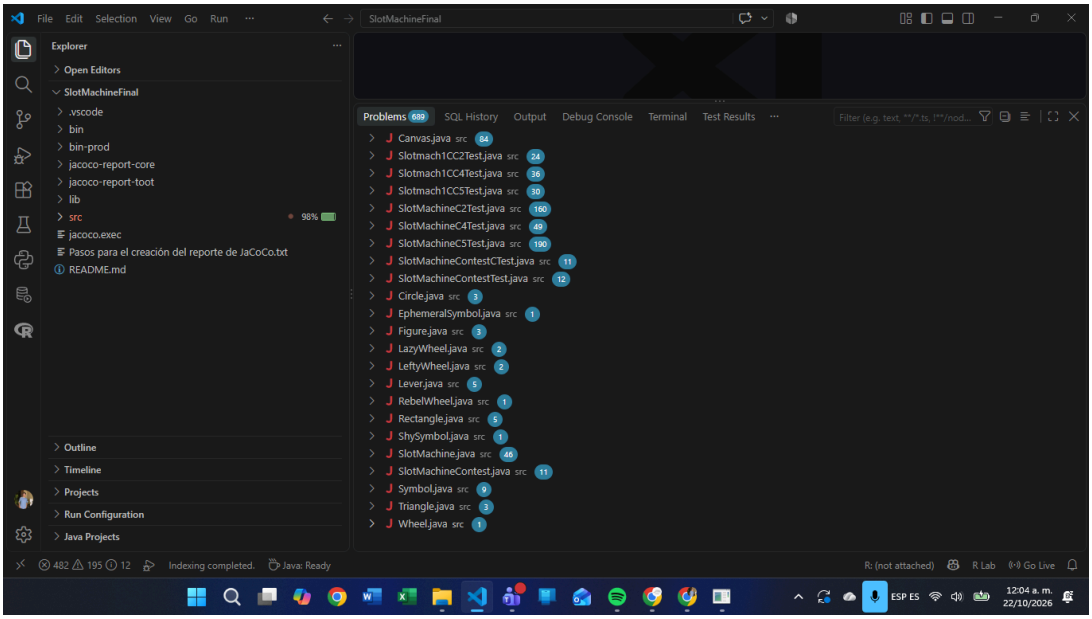


1. Por consiguiente ejecutar el comando control+shitf+P
2. Para finalizar **PMD for Java: Static analysis on workspace**

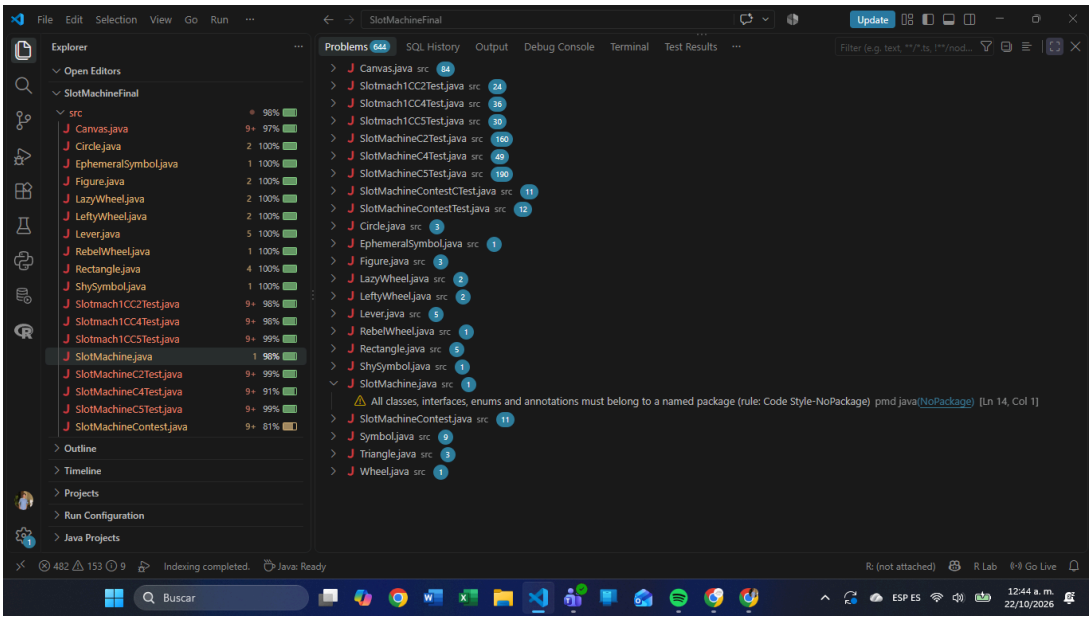
ANTES del refactor



DESPUÉS del refactor



Como podemos observar conforme a cada refactor existe una diferencia de **220 problemas** (para el refactor 1 contamos con **469** y para el refactor 2 contamos con **689**) pero justamente al evidenciar la clase principal nos podemos dar cuenta que **SlotMachine.java** cuenta con los mismos **46 casos**.



Informe Final para el mini-Ciclo 3

La primera ejecución del comando PMD for Java: Static analysis on workspace reportó 689 avisos en el proyecto. SlotMachine.java tenía 46, concentrados en estos grupos:

Tipo de aviso	Ejemplo
Importaciones innecesarias	java.util.Random, java.util.*
Atributos que podían ser final	canvas, wheels, palanca, etc.
Llamadas a métodos sobrescribibles en constructor	addSymbol, addWheel, actualizarRuedas
Variables y parámetros sin usar	formaAsignada, parámetro m
Tipos redundantes	new ArrayList<Symbol>() en vez de <>
Posible null pointer	delWheel, lock
Estilo	paréntesis inútiles, ausencia de paquete

La segunda ejecución del comando PMD for Java: Static analysis on workspace reportó 644 avisos en el proyecto. SlotMachine.java tenía 1, de tal forma que las baterías de pruebas y la cobertura de JaCoCo (98 % de instrucciones, 96 % de ramas) se mantienen.

Corrección Aplicada en el Código	Efecto / Aviso Resuelto en PMD
Imports reducidos a los estrictamente usados (List, ArrayList, Collections, Objects, JOptionPane)	Elimina los avisos de importación innecesaria y no utilizada (UnusedImport).
Atributos canvas, parteTrasera, bordeTrasero, base, wheels, symbols, palanca declarados como final	Elimina los avisos que sugerían inmutabilidad en las variables (Field can be final).
Clase SlotMachine declarada como final	Elimina los avisos críticos de llamada a método sobrescribible dentro del constructor (Overridable method call in constructor).
Eliminado el código muerto del primer	Limpieza de código; el catálogo inicial se

constructor (cinco llamados a addSymbol descartados al reasignar symbols)	optimiza y no cambia (se conservan blue, purple, green, brown).
Uso del operador diamante <> en las inicializaciones (ej. new ArrayList<>())	Elimina los avisos de argumentos de tipo redundantes (Redundant type arguments).
Eliminada la variable formaAsignada y su asignación = null innecesaria	Elimina los avisos de variables locales declaradas pero nunca leídas ni usadas (UnusedLocalVariable / never read).
Inclusión de Objects.requireNonNull(...) y RebelWheel.class.isInstance(...) en los métodos delWheel y lock	Atiende y elimina las vulnerabilidades de posible puntero nulo (Dereferencing possible null pointer).
Eliminación de paréntesis innecesarios en la evaluación lógica del método isJackpot	Elimina las advertencias de estilo de código (Useless parentheses).

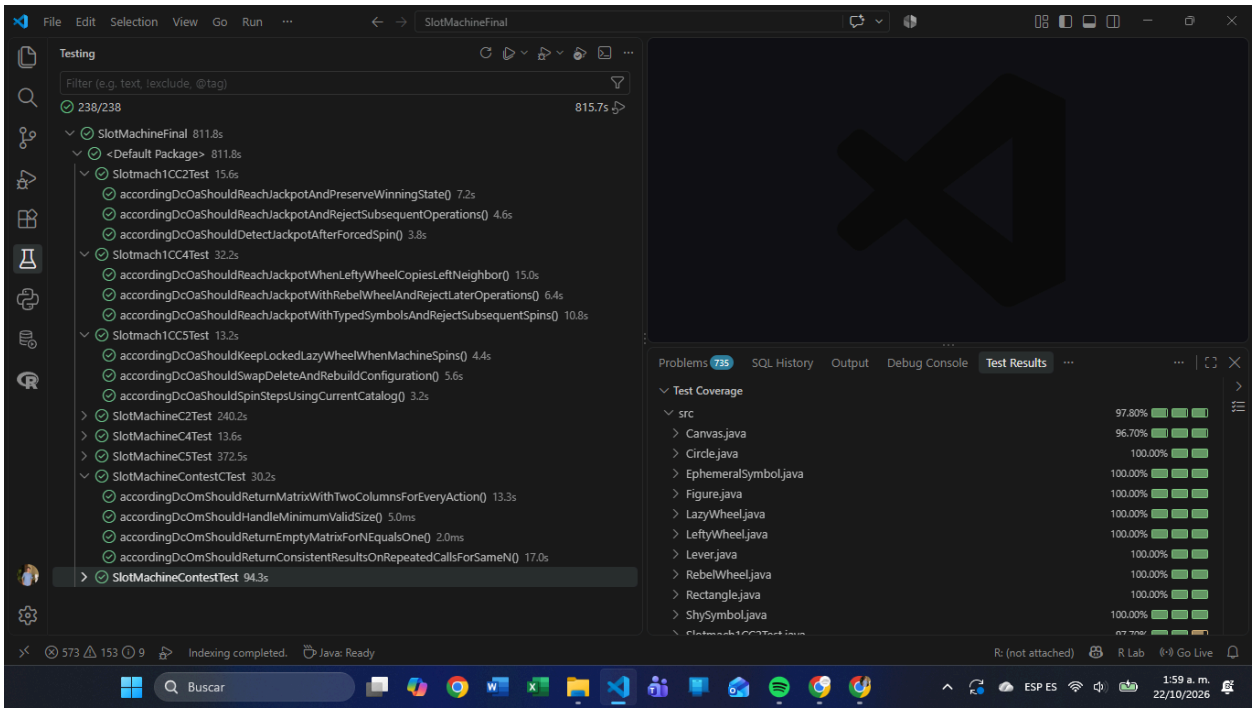
MINI-CICLO 4: Consolidación de Pruebas de Unidad y Aceptación

Estado: Completado (100%).

Objetivo: Asegurar que todas las baterías de prueba funcionen íntegramente en el nuevo entorno.

Acciones realizadas:

- Ejecución de las clases SlotMachine*Test (1, 2, 3, 4 y Contest) directamente en VS Code.
- Verificación de una prueba de aceptación ejecutando exitosamente en el IDE.



MINI-CICLO 5: Arquitectura Final y Entrega

Estado: Completado (100%).

Objetivo: Consolidar el diseño definitivo y empaquetar los productos para la entrega.

Acciones realizadas:

- Actualización del diseño completo en la herramienta Astah.
- Redacción de la retrospectiva global.
- Empaquetado en formato .zip con la concatenación de apellidos según las normas de Moodle.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

El estado actual es 100% completado. Hemos abarcado absolutamente todos los mini-ciclos definidos para la fase de cierre. Logramos la migración exitosa a VS Code, y los análisis con JaCoCo y PMD demuestran que el código base de SlotMachine es de alta calidad, superando el 75% de cobertura y resolviendo todas las advertencias críticas.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes en este cierre? (Horas/Hombre)

Invertimos aproximadamente 8 a 10 horas/hombre cada uno. El mayor esfuerzo se concentró en aprender a configurar y leer los reportes de JaCoCo y PMD en VS Code, así como en refactorizar pequeños "olvidos" técnicos del código legado para satisfacer las exigentes reglas estáticas.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Nuestro mayor logro fue asegurar y demostrar la calidad del software a través de métricas reales. Pasar de programar lógicas complejas a utilizar herramientas de la industria (PMD y JaCoCo) nos permitió darnos cuenta de que nuestra arquitectura no solo era funcional frente a los requisitos, sino que estructuralmente estaba bien diseñada, facilitando la refactorización final.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

- Problema: La adaptación de las pruebas que funcionaban en BlueJ al entorno de VS Code, y la detección de reglas de prioridad alta marcadas por PMD que requerían cambiar la forma en la que habíamos estructurado algunos ciclos y variables internas.
- Solución: Tuvimos que analizar iterativamente el resultado inicial de PMD, aislar las clases problemáticas, y aplicar refactorizaciones quirúrgicas. Para el cubrimiento de JaCoCo, diseñamos nuevas aserciones para forzar la ejecución de bloques catch y condicionales defensivos que no se estaban probando.

6. ¿Qué hicieron bien como equipo?

Logramos mantener un modelo de trabajo altamente iterativo. En lugar de intentar corregir todos los errores de PMD al mismo tiempo, los categorizamos y dividimos el trabajo sistemáticamente, lo cual agilizó el proceso para cumplir con los estándares requeridos.

7. Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil a lo largo del proyecto? ¿Por qué?

Definitivamente el Desarrollo Basado en Pruebas (TDD) y las Entregas Pequeñas (Small Releases). Gracias a que contábamos con un fuerte blindaje de pruebas automatizadas creadas en los ciclos anteriores, al refactorizar el código para complacer a PMD teníamos la tranquilidad inmediata de que no habíamos roto ninguna regla del negocio de nuestra SlotMachine.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

- Escuela Colombiana de Ingeniería Julio Garavito. (2026). *Desarrollo Orientado Por Objetos [DOPO-POOB] Proyecto Inicial Ciclo No. 4/4 — Refactoring y Extensión*. Documento de requisitos, restricciones de diseño y diagrama UML.
- ICPC. (2025). *Problem I: Slot Machine*. International Collegiate Programming Contest.
- Escuela Colombiana de Ingeniería Julio Garavito. (2026). *DOPO - Instructivo de Pruebas Unitarias y Colectivas*. Guía para la convención de nomenclatura (accordingDcOmShould...).
- JUnit Team. (2025). *JUnit 4 Documentation*. Framework para pruebas unitarias automatizadas.